FederHub Platform User Stories

Overview:

This document presents the unified user stories for the FederHub project, focusing on the key personas: Platform Admin, ML Engineer (Federation Owner), and Client Operator. The platform supports privacy-preserving machine learning through decentralized orchestration, secure data handling, and metric monitoring.

Structured under 4 epics, each story adheres to the INVEST framework, MOSCOW prioritization, T-shirt sizing, and the 3 C's for agile development.

Epic 1: Platform Management and Authentication

Focus: Organization onboarding, access control, and platform-wide security.

Story 1.1 As a Platform Admin, I want to manage organizations and users (view requests, add, remove, assign roles), so that platform access is controlled and secure.

- **Acceptance Criteria:**
  - If the user request is accepted, then the user should be able to log in successfully with their unique, secure credentials.
  - If a deleted user attempts to log in, then authentication instantly fails and access is denied.
- **Priority:** Must
- **Sizing:** M
- **3 C's:**
  - Card: This story card.
  - Conversation: Discuss security tiers and JWT lifecycle.
  - Confirmation: Verify organization creation and test user deactivation.
- **INVEST:**
  - Independent: Can be built largely on its own (org/user CRUD + role assignment) without needing the federated training features first.
  - Negotiable: The exact onboarding flow can vary (invite link, admin-created credentials, SSO, user account requests)
  - Valuable: Directly improves security and control—only approved users get access and roles restrict what they can do

    - Estimable: Work is predictable (standard database tables + REST APIs + UI screens + auth checks), so effort can be estimated.
    - Small: Can fit in a sprint if scoped to essentials (create org, add/remove users, assign roles, deactivate users).
    - Testable: Easy to verify with tests. Create org/user- login works; deactivate user - login fails; role restrictions → correct permissions

Story 1.2 As a user, I should be able to register for a new account and select my role. I should also be able to view the current status of my account

- **Acceptance Criteria:**
    - When the user submits the registration form with a valid email, password and selects a role, then the request is sent to the Federation owner and the status shows Pending Approval (or the user logs in if auto-approval is enabled for client operators).
    - When the user tries to register with an email that already exists, then the system rejects it and shows a clear error message.
- **Priority:** Must
- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Discuss which roles a user is allowed to choose at sign-up (e.g., Client Operator, ML Engineer), whether role requests need admin approval, and what statuses exist (Pending/Active/Rejected/Deactivated). Also discuss password rules and whether email verification is required for MVP.
    - Confirmation: Register a new user with a role and verify that status shows correctly. Try duplicate email and verify that the requests are blocked.
- **INVEST:**
    - Independent: Can be implemented without federated training features; only needs basic user/org management + authentication.
    - Negotiable: Approval workflow (auto-approve vs manual approve), allowed roles at signup, and email verification can change without breaking the story.
    - Valuable: Enables onboarding new users and makes the platform usable by real organizations while keeping access controlled.
    - Estimable: Standard registration UI + backend endpoints + status field + authorization checks; effort is predictable.
    - Small: MVP scope is limited to register/login, choose role, and view status page (no advanced profile management)

    - Testable: Can be tested with clear scenarios: successful registration, duplicate registration rejection, status display, and access control based on status

Story 1.3 As a user, I should be able to log in to my account if my account has been approved and I should be allowed to carry out the tasks intended with my role.

- **Acceptance Criteria:**
    - If the user's email ID or password are incorrect, then the system shows an error.
    - If the credentials are correct, but the account is either pending approval or deactivated then the system shows a clear error message.
    - If the credentials are correct and the status is active then the user should log in to the system and be allowed to carry out the tasks pertaining to their role
    - If the user chooses forgot password, then the system should allow them to search for their account using an email ID and send them an OTP to verify their email address (May be omitted from the minimum viable product).
- **Priority:** Must
- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Discuss the authentication method (email+password for MVP), password rules, session handling (JWT expiry/refresh), and role-based authorization boundaries (what each role can/can't do after login). Also decide whether "forgot password via OTP" is in MVP or deferred, and what account statuses are supported (Pending/Active/Deactivated/Rejected).
    - Confirmation: Try logging in with wrong password and verify "incorrect credentials" error. Try valid credentials with Pending or Deactivated status and verify clear status-based denial message. Try valid credentials with Active status and verify login success and role-specific access (allowed pages work, restricted pages return 403/blocked UI). If OTP reset is included: trigger "forgot password" and verify OTP sent, OTP verification works, and password can be reset.

- **INVEST:**
    - Independent: Can be implemented once registration/user records exist; does not depend on training-job or dashboard features.
    - Negotiable: Forgot-password flow (OTP vs email link), session policy (JWT refresh vs re-login), and exact error messaging can be adjusted.
    - Valuable: Essential for any real system—enables secure access and ensures users only perform actions allowed for their role.

    - Estimable: Standard login + status checks + RBAC checks; well-known implementation effort and test surface.
    - Small: Fits in a sprint when scoped to login + status gating + basic role authorization (OTP reset optional).
    - Testable: Easily testable via clear cases: wrong credentials, correct-but-inactive status, correct-and-active access, and role-based authorization checks.

Epic 2: Job Definition and Experimentation

Focus: Hyperparameter configuration, model templates, and training orchestration.

Story 2.1 As an ML Engineer, I want to upload baseline model templates, so that the server has a starting point to distribute to clients.

- **Acceptance Criteria:**
    - Given a PyTorch model file, when uploaded to the registry, then it is tagged with a unique version ID and upload date
    - The model is also evaluated against a defined dataset and the metrics are displayed next to the model.
- **Priority:** Must
- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Discuss what file formats are allowed for "baseline templates" (e.g., .pt/.pth, TorchScript), maximum size limits, required metadata (model name, framework version, input shape), who is allowed to upload (ML Engineer only), and what "evaluation dataset" means for the platform (fixed internal sample dataset vs user-provided validation set). Also clarify whether evaluation runs synchronously at upload time or asynchronously as a background job, and what happens if evaluation fails.
    - Confirmation: Upload a valid PyTorch model file and verify the registry shows a unique version ID and upload date; verify evaluation runs and the resulting metrics appear beside the model. Upload an invalid/unsupported file and verify the system rejects it with a clear error. Trigger an evaluation failure (e.g., incompatible input shape) and verify the model is still stored but marked "Evaluation Failed" with an error message (or the agreed policy for MVP).

- **INVEST:**
    - Independent: This story can be built without implementing full federated training rounds because it only requires a model registry and an evaluation pipeline; federation distribution can consume the uploaded model later.
    - Negotiable: The evaluation approach can change (fixed dataset vs configurable dataset, immediate vs async evaluation), and even the model packaging method can evolve, as long as the platform supports uploading a baseline and attaching metrics.
    - Valuable: It ensures the server starts from a known, versioned baseline model and gives the ML Engineer confidence via displayed metrics, which improves reproducibility and avoids "unknown starting point" issues.
    - Estimable: The work is predictable: file upload + storage, metadata/versioning, and a straightforward evaluation job with metric logging and UI display.
    - Small: It fits within a sprint when scoped to PyTorch-only upload, basic metadata/versioning, and one evaluation dataset with a small set of metrics; advanced features (multi-framework, large-scale evaluation, model cards) can be deferred.
    - Testable: It's testable through clear checks: version ID and upload date are created on upload, metrics appear after evaluation, unsupported files are rejected, and failure scenarios produce an explicit "evaluation failed" state and logs.

Story 2.2 As an ML engineer, I should be able to view incoming model push requests, test the model on a defined dataset, reject/accept the request, add models to platform logs along with its version number, date, and performance metrics, and remove/edit existing model versions.

- **Acceptance Criteria:**
    - Given there is a pending model push request, when the ML engineer opens the "Model Requests" page, then the ML engineer can view request details (requester, model name, file type/size, submitted timestamp) and the request status is clearly shown as Pending.
    - Given the ML engineer selects a pending request, when the ML engineer clicks "Run Evaluation," then the system evaluates the model on the defined dataset and displays performance metrics (e.g., accuracy/loss) and an evaluation status (Completed/Failed) next to the request. (The evaluation can be local as well)
    - Given evaluation has completed successfully, when the ML engineer clicks "Accept," then the model is added to the model registry/logs with a unique

version number, upload/approval date, and the displayed metrics, and the request is removed from the pending requests page.
    - Given evaluation is completed (or failed), when the ML engineer clicks “Reject,” then the request status becomes Rejected and the model is not added to the registry/logs.

- **Priority:** Should
- **Sizing:** L
- **3 C’s:**
    - Card: This story card.
    - Conversation: Define what fields a model push request must include, what “defined dataset + metrics” are used for evaluation, and whether “remove” means soft-deprecate or hard delete for MVP.
    - Confirmation: Pending requests appear with details and evaluation runs and metrics display. Verify that Accept adds a versioned entry (version/date/metrics) and Reject does not add entry (optional reason). Verify that edit metadata saves create a log entry.
- **INVEST:**
    - Independent: This story can be implemented without full federated orchestration because it mainly involves model request intake, evaluation, approval workflow, and model registry management; training jobs can later consume “approved” models.
    - Negotiable: The exact approval workflow can change (auto-approve for trusted users, optional evaluation, different metrics, different dataset, async evaluation), and “edit/remove” can be scoped to metadata or soft deletion depending on MVP constraints.
    - Valuable: It provides governance and quality control—only validated models enter the platform registry, and versioned logging improves reproducibility, traceability, and safe model reuse.
    - Estimable: The work is clear: request queue UI/API, evaluation job runner, accept/reject transitions, registry updates, and edit/remove operations with audit logging.
    - Small: It is larger than a simple story, but it can still be kept sprint-sized by scoping MVP to: view requests, run evaluation, accept/reject, and log version +metrics; advanced editing and deletion policies can be minimal (metadata edit + deprecate).
    - Testable: It is testable end-to-end with scenarios: pending request visibility, evaluation produces metrics, accept adds versioned registry entry, reject

prevents registry entry, and edit/remove actions persist with audit/log verification.

Story 2.3 As an ML engineer, the ML engineer should be able to start a federated orchestration job by selecting a baseline model version, so that the platform can distribute the model to clients, collect client updates each round, aggregate weights, and evaluate the final global model.

- **Acceptance Criteria:**
  - Given the ML engineer is on the "Orchestration" page, when the ML engineer selects a baseline model version from the model registry and clicks "Start Job," then the system creates a new job with status Running and shows the selected baseline model version in the job details.
  - Given a job is Running, when clients submit updates for a round, then the ML engineer can view each individual client submission (client ID, timestamp, update size, sample count if available, and submission status).
  - Given at least the configured minimum number of client submissions are received (or the round timeout occurs), when the system triggers aggregation, then the system aggregates the received weights (FedAvg for MVP) and the UI shows the aggregation event and the resulting global model version for that round.
  - The ML engineer should be able to evaluate the aggregated model on a preset dataset to determine its performance and choose if the model is to be added or not.
- **Priority:** Should
- **Sizing:** L
- **3 C's:**
  - Card: This story card.
  - Conversation: Confirm job configuration fields needed for MVP (baseline model version, number of rounds, minimum clients, timeouts), what "individual submissions" metadata is stored/displayed, which aggregation method is used (FedAvg), and whether evaluation runs only at the end or after every round.
  - Confirmation: Start a job with a selected baseline model and verify clients receive the model and submissions appear. Verify aggregation happens when threshold/timeout is reached. Verify round progress is visible. Verify final evaluation runs and metrics .

- **INVEST:**
    - Independent: Depends on model registry, client registration, and basic update submission endpoints, but can be implemented without advanced features like alerts or template reuse.
    - Negotiable: Aggregation algorithm, evaluation frequency (end-only vs per-round), and the exact submission metadata displayed can evolve without changing the core value of the story.
    - Valuable: Enables the primary user workflow—starting federated training, monitoring participation, producing a validated final global model.
    - Estimable: Work items are clear: job creation, state transitions, per-round submission tracking, aggregation trigger + artifact versioning, and evaluation pipeline + UI.
    - Small: This is larger than a single sprint story, but can be split into MVP increments (start job + track submissions, then aggregation, then final evaluation + registry).
    - Testable: Testable via integration tests: start job with chosen baseline, simulate client submissions, verify aggregation output creation, verify final evaluation metrics are produced and stored.

Epic 3: Privacy and Security

Focus: Ensure sensitive data is not centralized, never leaked, never leaves the local machine, and kept private.

Story 3.1 As a Client Operator, I want to make sure that my sensitive data never leaves my local machine and is kept confidential.

- **Acceptance Criteria:**
    - Given a client is participating in a federated training job, then the client application only transmits model updates (e.g., weights/gradients and allowed metadata like sample count) to the server and never transmits raw dataset files or raw records.
    - Given the client is training an available model with a configured path of their dataset, then the dataset is accessed only from local storage and the system does not upload, sync, or store any dataset contents on the server. Only the trained model is transmitted/uploaded to the server (either manually by the client on their website or automatically by the system) for the ML engineer to review.
- **Priority:** Must

- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Agree on exactly what can be sent to the server (model weights/gradients, sample count, training metrics) and what is forbidden (raw files, raw records, identifiers). Clarify the "trained model upload" step: whether the client uploads a full trained model artifact or only updates, and whether that upload is manual or automatic. Confirm encryption expectations and what logs/indicators the client operator should see to build trust.
    - Confirmation: Run a job and verify that only model updates or approved model artifacts are sent and no dataset content is transmitted. Check that the dataset is read from a local path only. Verify that the server receives a model update or trained model artifact that the ML engineer can review, and verify that raw data upload attempts are rejected or blocked.

- **INVEST:**
    - Independent: This can be implemented with client-side restrictions and server-side validation without needing the full orchestration UI or advanced monitoring features.
    - Negotiable: The exact enforcement method can change, and the platform can evolve from manual model upload to automatic submission without changing the core privacy goal.
    - Valuable: This is the main reason organizations will trust and adopt federated learning, because it protects sensitive data.
    - Estimable: The work is clear and standard: payload rules, transport security, and simple checks/logs on both client and server.
    - Small: With MVP scope limited to blocking raw data transfer and allowing only model updates or approved artifacts, this fits a sprint.
    - Testable: It can be tested by inspecting outgoing requests, verifying local-only data access, confirming the server never receives raw data, and validating that only approved model submissions are accepted.

Epic 4: Client Accessibility

Focus: Facilitate smooth access of all supported functionalities to the client.

Story 4.1 As a client operator, I should be able to see a dashboard of all available models, along with their version number, performance, and date, so that I can choose the model that works best with my data.

- **Acceptance Criteria:**
    - Given the client operator opens the “Models” dashboard, then the system displays a list of available models with model name, version number, upload/approval date, and performance metrics on the predefined dataset.
    - Given multiple model versions exist, when the client operator sorts or filters (by date, version, or performance), then the list updates accordingly.
- **Priority:** Should
- **Sizing:** M
- **3 C’s:**
    - Card: This story card.
    - Conversation: Decide what “available models” means for a client operator (all public models vs only models approved for their organization or job). Confirm which metrics to show, how to label the evaluation dataset, and whether selection means “download for local testing” or “choose for a job request.”
    - Confirmation: Open the models dashboard and verify all required fields display. Apply sort/filter and verify results change correctly. Select a model version and verify the system records the selection and starts the allowed next step (download/use in local run or request participation).

- **INVEST:**
    - Independent: Can be built once the model registry exists; it does not require building full federated orchestration first.
    - Negotiable: Filters, metrics displayed, and the exact “select model” action can evolve without changing the core goal of model discovery.
    - Valuable: Helps client operators make informed choices and reduces trial-and-error by showing versioned performance information.
    - Estimable: Standard dashboard work with known scope: query models endpoint, render table, sorting/filtering, and selection action.
    - Small: Small if scoped to an MVP dashboard that only shows a table of models (name, version, date, key metrics) plus basic sort/filter and a “select” action. Advanced features like per-client performance, charts, or model comparisons can be separate stories.
    - Testable: Tests can verify the API returns the correct fields, the UI renders them, sorting/filtering changes the ordering correctly, and selecting a model saves the chosen model/version for that client (and blocks selection if the role/permissions don’t allow it).

Story 4.2 As a client operator, I should be able to push the trained model to the server. It may be a standalone model push or I may be participating in an orchestrated training round.

- **Acceptance Criteria:**
    - Given the user has completed local training in standalone mode, when the user clicks "Push Model," then the system uploads the trained model artifact (or approved update payload) to the server, creates a new submission entry with timestamp, model version reference, and submission status, and shows a success message if upload completes.
    - Given the user is participating in an orchestrated training round, when local training for that round finishes, then the client automatically submits the required model update payload to the server within the round window and the server records the submission under that job and round.
    - Given a submission is completed, when the user views submission history, then the user can see the submission status and metadata (job/round if applicable, submission time, payload size).
- **Priority:** Must
- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Clarify what exactly is pushed in each case (full trained model artifact vs weight/gradient update) and which roles can do it. Confirm file format limits, encryption requirements, and how submissions are linked to either a standalone push request queue or a specific orchestration job/round. Decide the retry policy and what metadata is stored for audit/debugging.
    - Confirmation: Complete a standalone local training run and push the model, then verify the server creates a submission entry and the UI shows success. Run an orchestrated round and verify the automatic submission is recorded under the correct job and round. Simulate a failure (invalid payload or network) and verify clear error + retry works and submission status is not marked successful.
- **INVEST:**
    - Independent: Standalone push can be implemented with a submission endpoint and storage even if full orchestration is not complete; orchestration-linked submissions can reuse the same submission pipeline.

    - Negotiable: The payload type can evolve (artifact vs update), and the submission UX can vary (manual button vs fully automatic) while preserving the core requirement to deliver results to the server.
    - Valuable: Enables the platform's core collaboration loop by getting client-trained updates/models to the server for review, aggregation, and versioning.
    - Estimable: Standard work with known scope: upload API, validation, storage, status tracking, and simple UI feedback.
    - Small: Fits a sprint if scoped to one supported payload type, basic metadata, and retry; deeper features like delta compression or streaming can be future stories.
    - Testable: Can be tested by verifying successful upload creates a submission record, orchestrated submissions attach to the correct job/round, invalid payloads are rejected, failures allow retry, and submission history shows correct statuses.

Story 4.3 As a client operator, I should be able to test/train the model on my local dataset, with minimal knowledge of machine learning using easy to use UI.

- **Acceptance Criteria:**
    - Given the user downloaded their desired model, when the user runs the provided local training/testing flow (e.g., a simple UI runner or one command with prompts), then the model is trained/tested locally using the dataset path provided by the user.

- **Priority:** Must
- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Confirm what exactly gets downloaded (model only vs model + runner), what "easy-to-use" means (one-click local UI vs one command), the expected dataset format, and which minimal metrics should be shown. Also decide whether the website needs to show "copy-paste command" instructions or provide a bundled local tool.
    - Confirmation: Download a model from the website and verify the correct model version is received. Run the local flow with a valid dataset and confirm it completes and shows metrics. Run with an invalid dataset path and confirm execution is blocked with a clear fix message. Confirm the local run record stores model version + timestamp + metrics.

- **INVEST:**
    - Independent: Can be implemented once the model registry + download endpoint exist; it does not require federated orchestration to be complete.
    - Negotiable: The local experience can vary (UI runner vs CLI prompts) and metrics can evolve, while keeping the core requirement of "download then local train/test."
    - Valuable: Enables non-ML users to actually use models from the platform and evaluate on their own local data without coding ML pipelines.
    - Estimable: Website side is a standard download workflow; local side is a simple guided runner with validation + basic reporting.
    - Small: MVP can support one dataset format, one model type, and a minimal local runner; advanced tuning and visualization can be separate stories.
    - Testable: estable by verifying download correctness, dataset validation behavior, successful local run behavior, and result logging with model version metadata

Story 4.4 As a client operator, I want to make sure that along with the baseline model, I get adequate documentation, including the format of the dataset (features, format etc.) for correct training and testing.

- **Acceptance Criteria:**
    - When the user selects the model version to use, then a clear documentation should be provided including model details, dataset requirements, file type, folder structure, required columns/features, label format, and any preprocessing expectations. The documentation should also include operational instructions for the user on how they should use the system. This documentation can either be provided via a Readme File along with the model or as readable text on the system under model details.

- **Priority:** Must
- **Sizing:** M
- **3 C's:**
    - Card: This story card.
    - Conversation: Agree on what "adequate documentation" must include for MVP, which dataset formats are supported, and what operational instructions are required (download steps, how to run train/test locally, how to push updates if applicable). Decide whether the documentation is stored as a README bundled with the model download, as text on the model details

page, or both. Confirm that documentation is tied to the selected model version so users don't follow the wrong instructions.
    - Confirmation: Select a model version and verify the documentation is immediately visible and includes all required sections (model details, dataset format, preprocessing, operational steps). Download the model and verify documentation is available in the chosen form (README file or model details page). Verify that selecting a different model version shows the correct version-specific documentation.

- **INVEST:**
    - Independent: This can be implemented once model versions exist in the platform and does not depend on federated orchestration or client training logic.
    - Negotiable: The exact format and delivery of documentation can change (README, web text, PDF) and the level of detail can grow, while keeping the core requirement of clear dataset and usage instructions.
    - Valuable: It prevents users from training/testing incorrectly, reduces confusion for non-ML users, and improves successful adoption of the platform.
    - Estimable: The work is straightforward: define a documentation template, store it per model version, render it on the model page and/or bundle it with downloads.
    - Small: With MVP scope limited to a fixed template and one supported dataset format, this fits within a sprint; richer tutorials and troubleshooting guides can be separate stories.
    - Testable: It can be tested by checking required documentation fields exist for each model version, verifying the documentation is accessible at selection and download, and verifying the content shown matches the selected version.